Лекция № 17
Контроль остатков материалов при проведении документа.

ТЕОРЕТИЧЕСКАЯ ЧАСТЬ

Оперативное и неоперативное проведение документов

При разработке конфигураций на платформе «1С:Предприятие» используется концепция оперативного и неоперативного проведения документов. Эта концепция используется при решении задач оперативного учета (например, при складском учете товаров) и позволяет организовать корректную работу с документами в условиях реальной действительности. Однако можно отключить этот механизм (запретить оперативное проведение документов) и реализовывать собственные алгоритмы, проверять актуальность документов на оси событий и т. д.

Оперативное проведение документов пользователями выполняется **в режиме «реального времени»,** то есть отображает изменения, факты, свершающиеся в настоящее время. Оперативное проведение особенно актуально при многопользовательской работе. Поэтому при этом способе проведения документов следует осуществлять максимум проверок, способных исключить ошибки при вводе данных пользователями.

Например, при оперативном проведении следует выполнять контроль остатков на складе списываемой номенклатуры с тем, чтобы исключить одновременную продажу одного товара несколькими продавцами.

При оперативном проведении документа система, прежде всего, проверит положение даты документа относительно текущей даты сеанса. Текущая дата сеанса равна системной дате компьютера, приведенной к часовому поясу сеанса. Если дата проводимого документа совпадает с текущей датой сеанса, то система будет проводить такой документ в оперативном режиме, и в обработке проведения об этом можно узнать, чтобы выстроить определенный алгоритм проведения документа.

Если дата проводимого документа меньше текущей даты сеанса, то такой документ система будет проводить в неоперативном режиме.

Неоперативное проведение документов подразумевает отражение в базе данных фактов, которые свершились в прошлом или которые точно будут совершены в будущем. Поэтому **задача неоперативного проведения** документов — просто отразить в информационной базе данные о совершенных операциях.

При неоперативном проведении документов не имеет смысла производить целый ряд проверок, в частности контроль остатков. Подразумевается, что если в процессе неоперативного проведения документов были допущены ошибки (например, списано такое количество номенклатуры, которого не было на складе на дату проведения документа), то анализ полученного состояния базы данных является отдельной задачей, не относящейся к неоперативному проведению и выполняющейся не в момент проведения документа, а тогда, когда в базе имеются достаточные данные для анализа, например, когда введены более ранние документы, приходующие товары.

Таким образом, **оперативное проведение** служит для того, чтобы **в реальном режиме** многопользовательской работы определить возможность или невозможность выполнения той или иной операции (и выполнить ее, если возможно).

Неоперативное проведение предназначено **для безусловного отражения в базе операций**, которые уже были совершены (или точно будут совершены).

Контроль остатков

При проведении документа **ОказаниеУслуги** необходимо **контролировать остатки расходуемых товаров на складе**. Если при проведении документа **ОказаниеУслуги** товаров на складе не хватает, надо сообщить пользователю и не проводить документ.

Общая методика контроля остатков при проведении документа заключается в следующем: сначала, не глядя ни на что, нужно записать движения документа, а затем, когда движения уже записаны, прочитать из базы данных остатки.

Если появились отрицательные остатки, значит, такой документ проводить нельзя. Нужно сообщить пользователю, каких материалов не хватает, и отменить проведение документа.

Если же отрицательных остатков не появилось, тогда можно смело проводить документ.

Полдела у нас уже сделано: мы формируем движения документа и записываем их. Единственное, что нам осталось, — в случае **оперативного проведения** проконтролировать, что получилось, и, если появились отрицательные остатки, отменить проведение документа.

ПРАКТИЧЕСКАЯ ЧАСТЬ

- ***В режиме Конфигуратор***

Сделаем заготовку. После цикла обхода результата запроса и перед концом процедуры напишем следующие строки (листинг 17.1).

Листинг 17.1. Фрагмент процедуры «**ОбработкаПроведения**()»

```
…
  КонецЦикла;

  Движения.Записать();
  Если Режим = РежимПроведенияДокумента.Оперативный Тогда
    // Проверить отрицательные остатки
  КонецЕсли;

КонецПроцедуры
```

Сначала мы записываем движения в регистры.

Затем определяем режим проведения документа. При выполнении процедуры **ОбработкаПроведения()** вторым параметром (**Режим**) в нее передается режим проведения документа, и значение этой переменной сравнивается со значением системного перечисления **РежимПроведенияДокумента**. В случае оперативного проведения мы будем выполнять контроль остатков.

Теперь сделаем заготовку запроса для проверки отрицательных остатков.

Так как нам придется снова получать остатки только для той номенклатуры, которая в документе, укажем, что этот запрос будет использовать тот же самый менеджер временных таблиц **МенеджерВТ** (листинг 17.2).

Листинг 17.2. Фрагмент процедуры «**ОбработкаПроведения()**»

```
Если Режим = РежимПроведенияДокумента.Оперативный Тогда
  // Проверить отрицательные остатки
  Запрос3 = Новый Запрос;
  Запрос3.МенеджерВременныхТаблиц = МенеджерВТ;
  Запрос3.Текст = "";

КонецЕсли;
```

Установим курсор внутрь кавычек и вызовем конструктор запроса. Подтвердим создание нового запроса.

Выберем таблицу **ОстаткиМатериалов.Остатки** и из нее два поля: **Материал** и **КоличествоОстаток**.

Зададим параметры этой таблицы. В параметре **Условие** напишем:

Листинг 17.3. Условие виртуальной таблицы

```
Материал В (
  ВЫБРАТЬ
    НоменклатураДокумента.Номенклатура
  ИЗ
    НоменклатураДокумента)
И Склад = &Склад
```

То есть мы получаем итоги только для той номенклатуры, которая содержится в нашей временной таблице, и только по складу, который указан в документе.

Затем на закладке **Условия** перенесем в список условий поле **КоличествоОстаток**, установим флажок **Произвольное** и укажем, что нас интересуют только отрицательные остатки (листинг 17.4).

Листинг 17.4. Условие запроса

```
ОстаткиМатериаловОстатки.КоличествоОстаток < 0
```

Нажмем ОК. Текст запроса будет выглядеть следующим образом (листинг 17.5).

Листинг 17.5. Текст запроса

```
// Проверить отрицательные остатки
Запрос3 = Новый Запрос;
Запрос3.МенеджерВременныхТаблиц = МенеджерВТ;
Запрос3.Текст = "
  |ВЫБРАТЬ
  | ОстаткиМатериаловОстатки.Материал КАК Материал,
  | ОстаткиМатериаловОстатки.КоличествоОстаток КАК КоличествоОстаток
  |ИЗ
  | РегистрНакопления.ОстаткиМатериалов.Остатки( , Материал В (
  | ВЫБРАТЬ
  | НоменклатураДокумента.Номенклатура
  | ИЗ
  | НоменклатураДокумента)
  | И Склад = &Склад) КАК ОстаткиМатериаловОстатки
  |ГДЕ
  | ОстаткиМатериаловОстатки.КоличествоОстаток < 0";
```

Теперь осталось только установить параметр запроса, обойти результат запроса и вывести сообщения об отрицательных остатках (листинг 17.6).

Листинг 17.6. Фрагмент процедуры «ОбработкаПроведения()»

```
// Проверить отрицательные остатки
Запрос3 = Новый Запрос;
Запрос3.МенеджерВременныхТаблиц = МенеджерВТ;
Запрос3.Текст = "
  |ВЫБРАТЬ
  | ОстаткиМатериаловОстатки.Материал,
…
  |ГДЕ
  | ОстаткиМатериаловОстатки.КоличествоОстаток < 0";

Запрос3.УстановитьПараметр("Склад", Склад);
РезультатЗапроса = Запрос3.Выполнить();
ВыборкаДетальныеЗаписи = РезультатЗапроса.Выбрать();
Пока ВыборкаДетальныеЗаписи.Следующий() Цикл
  Сообщение = Новый СообщениеПользователю();
  Сообщение.Текст = "Не хватает " + Строка(-
  ВыборкаДетальныеЗаписи.КоличествоОстаток) + " единиц материала """ +
  ВыборкаДетальныеЗаписи.Материал + """";
  Сообщение.Сообщить();
  Отказ = Истина;
КонецЦикла;
```

Кратко поясним добавленный текст.

При выполнении проверки в запрос в параметре **Склад** передается склад, указанный в документе.

Затем выполняется запрос для получения отрицательных остатков номенклатуры, содержащейся во временной таблице и на складе, указанном в параметре **Склад**.

После этого выборка записей запроса обходится в цикле, и, если есть такие записи, они выводятся в сообщениях пользователю.

При этом параметру **Отказ** процедуры проведения документа присваивается значение **Истина**, то есть документ не проводится, начатая транзакция отменяется, и состояние данных, измененных в процессе проведения, возвращается в исходное, до начала проведения документа.

Блокировка данных, которые читаются и изменяются при проведении

Казалось бы, это все? Но нет, есть очень важный момент, о котором мы не позаботились.

Сейчас схема нашей процедуры такова:

1. Выполняем первый запрос с именем **Запрос**. В результате мы формируем временную таблицу из перечня номенклатуры документа.
2. Выполняем второй запрос с именем **Запрос2**. В результате мы читаем стоимость и остатки для номенклатуры, содержащейся в табличной части документа.
3. Записываем движения регистров (**Движения.Записать()**).
4. Выполняем третий запрос с именем **Запрос3**. Тем самым мы проверяем наличие отрицательных остатков.

Обратите внимание, что, начиная с выполнения второго запроса и до конца процедуры, нам необходимо обеспечить неизменность стоимости и остатков номенклатуры, с которой мы работаем, и запретить другим транзакциям даже читать эти данные. Сама система, естественно, заблокирует изменение этих данных, но лишь начиная с того момента, когда мы запишем движения.

Однако может возникнуть следующая ситуация. Выполняя второй запрос, мы прочитали, что есть 2 шт. некоторого материала. И другая транзакция (другой пользователь), которая собирается списывать материалы, тоже прочитала, что есть 2 шт. этого материала. После этого мы записали

движения, и система заблокировала эти данные. Другая транзакция ждет, когда мы освободим данные. Мы провели документ, списали 2 шт. материала и освободили данные. Другая транзакция пытается тоже списать 2 шт. материала, но его уже нет!

Аналогичная ситуация может возникнуть и между п. 3 и п. 4, в результате чего контроль остатков будет работать неверно.

Поэтому, чтобы не происходило таких коллизий, нам необходимо заблокировать остатки от чтения другими транзакциями еще до выполнения второго запроса. То есть прежде чем читать что-то, что мы собираемся изменять, нужно запретить чтение этих данных другими транзакциями до тех пор, пока мы не закончим свои изменения (или пока не откажемся от проведения документа).

- ***В режиме Конфигуратор***

Как это сделать? Хороший вопрос. Давайте посмотрим на свойство **Режим управления блокировкой данных** нашей конфигурации. Оно установлено в значение **Управляемый** (рис. 17.7).

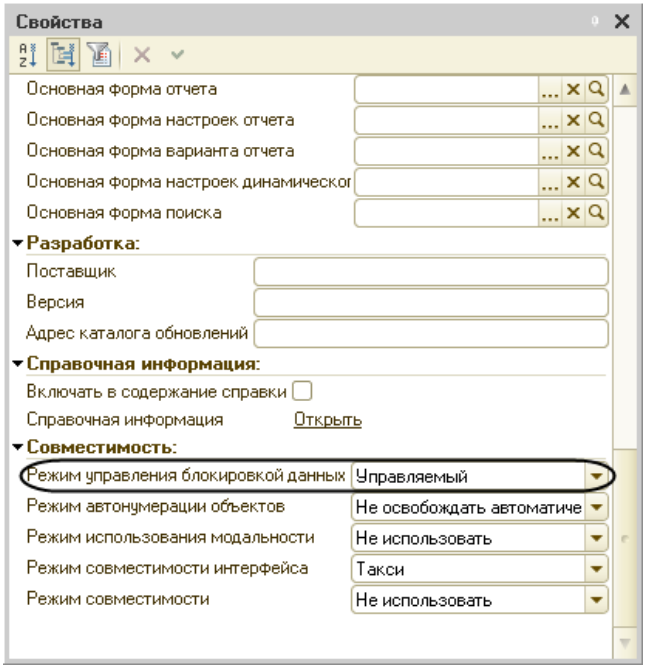


Рис. 17.7. Режим управления блокировкой данных в свойствах конфигурации

Это значит, что нам нужно использовать управляемые блокировки, которые устанавливаются средствами встроенного языка.

Необходимо заблокировать те данные, которые мы собираемся читать и впоследствии изменять. Для этого у наборов записей регистров есть свойство **БлокироватьДляИзменения**, которым мы и воспользуемся.

Вставим этот код перед записью пустых наборов записей (листинг 17.8).

Листинг 17.8. Фрагмент процедуры «**ОбработкаПроведения**()»

```
...
| ПО НоменклатураДокумента.Номенклатура = ОстаткиМатериаловОстатки.Материал";
// Установим необходимость блокировки данных в регистрах СтоимостьМатериалов и
ОстаткиМатериалов
Движения.СтоимостьМатериалов.БлокироватьДляИзменения = Истина;
Движения.ОстаткиМатериалов.БлокироватьДляИзменения = Истина;
// Запишем пустые наборы записей, чтобы читать остатки без учета данных в
документе
Движения.СтоимостьМатериалов.Записать();
Движения.ОстаткиМатериалов.Записать();
РезультатЗапроса = Запрос2.Выполнить();
```

Управляемая блокировка будет установлена в момент записи этих наборов записей, то есть как раз перед выполнением второго запроса. Что нам и требовалось.

Выделение произвольных областей модуля

Итак, мы закончили редактирование процедуры **ОбработкаПроведения**() в модуле документа ОказаниеУслуги. Хотя модуль содержит всего одну процедуру, эта процедура довольно объемная (особенно для неопытного разработчика), и ее нельзя «охватить одним взглядом». Это затрудняет процесс восприятия и понимания текста процедуры, поиск в ней ошибок и т. д.

Для удобства разработчика в редакторе модуля существует возможность выделять произвольные области текста, группировать и сворачивать их подобно тому, как сворачиваются инструкции циклов, условий, процедур и функций. Каждой области текста разработчик может дать собственное имя и таким образом выделить часть модуля, имеющую определенное назначение.

Например, в нашей процедуре можно выделить три логические части: в первой части формируется временная таблица, содержащая перечень номенклатуры документа; во второй части рассчитывается стоимость номенклатуры и формируются движения в регистрах накопления; и в третьей части производится контроль остатков номенклатуры при оперативном проведении документа.

Выделим эти три области в тексте процедуры, используя инструкции препроцессору **#Область <имя области>** и **#КонецОбласти** (листинг 17.9).

Листинг 17.9. Процедура «**ОбработкаПроведения**()»

```
Процедура ОбработкаПроведения(Отказ, Режим)

  Движения.ОстаткиМатериалов.Записывать = Истина;
  Движения.СтоимостьМатериалов.Записывать = Истина;
  Движения.Продажи.Записывать = Истина;

  // Создать менеджер временных таблиц
  МенеджерВТ = Новый МенеджерВременныхТаблиц;

  #Область НоменклатураДокумента
  Запрос = Новый Запрос;

  // Укажем, какой менеджер временных таблиц использует этот запрос
  Запрос.МенеджерВременныхТаблиц = МенеджерВТ;

  Запрос.Текст =
    "ВЫБРАТЬ
    | ОказаниеУслугиПеречеНьНоменклатуры.Номенклатура КАК Номенклатура,
    | ОказаниеУслугиПеречеНьНоменклатуры.Номенклатура.ВидНоменклатуры КАК ВидНоменклатуры,
    | СУММА(ОказаниеУслугиПеречеНьНоменклатуры.Количество) КАК КоличествоВДокументе,
    | СУММА(ОказаниеУслугиПеречеНьНоменклатуры.Сумма) КАК СуммаВДокументе
    |ПОМЕСТИТЬ НоменклатураДокумента
    |ИЗ
    | Документ.ОказаниеУслуги.ПеречеНьНоменклатуры КАК ОказаниеУслугиПеречеНьНоменклатуры
    |ГДЕ
    | ОказаниеУслугиПеречеНьНоменклатуры.Ссылка = &Ссылка
    |
    |СГРУППИРОВАТЬ ПО
    | ОказаниеУслугиПеречеНьНоменклатуры.Номенклатура,
    | ОказаниеУслугиПеречеНьНоменклатуры.Номенклатура.ВидНоменклатуры";

  Запрос.УстановитьПараметр("Ссылка", Ссылка);

  РезультатЗапроса = Запрос.Выполнить();
  #КонецОбласти

  #Область ДвиженияДокумента
  Запрос2 = Новый Запрос;
  Запрос2.МенеджерВременныхТаблиц = МенеджерВТ;
  Запрос2.Текст = "ВЫБРАТЬ
    | НоменклатураДокумента.Номенклатура КАК Номенклатура,
    | НоменклатураДокумента.ВидНоменклатуры КАК ВидНоменклатуры,
    | НоменклатураДокумента.КоличествоВДокументе КАК КоличествоВДокументе,
    | НоменклатураДокумента.СуммаВДокументе КАК СуммаВДокументе,
    | ЕСТЬNULL(СтоимостьМатериаловОстатки.СтоимостьОстаток, 0) КАК Стоимость,
    | ЕСТЬNULL(ОстаткиМатериаловОстатки.КоличествоОстаток, 0) КАК Количество
    |ИЗ
    | НоменклатураДокумента КАК НоменклатураДокумента
    | ЛЕВОЕ СОЕДИНЕНИЕ РегистрНакопления.СтоимостьМатериалов.Остатки(
    | ,
    | Материал В
    | (ВЫБРАТЬ
    | НоменклатураДокумента.Номенклатура
    | ИЗ
    | НоменклатураДокумента)) КАК СтоимостьМатериаловОстатки
    | ПО НоменклатураДокумента.Номенклатура =
    СтоимостьМатериаловОстатки.Материал
    | ЛЕВОЕ СОЕДИНЕНИЕ РегистрНакопления.ОстаткиМатериалов.Остатки(
    | ,
    | Материал В
    | (ВЫБРАТЬ
    | НоменклатураДокумента.Номенклатура
    | ИЗ
    | НоменклатураДокумента)) КАК ОстаткиМатериаловОстатки
    | ПО НоменклатураДокумента.Номенклатура =
    ОстаткиМатериаловОстатки.Материал";
```

```
  // Установим необходимость блокировки данных в регистрах СтоимостьМатериалов
  и ОстаткиМатериалов
  Движения.СтоимостьМатериалов.БлокироватьДляИзменения = Истина;
  Движения.ОстаткиМатериалов.БлокироватьДляИзменения = Истина;

  // Запишем пустые наборы записей, чтобы читать остатки без учета данных в
  документе
  Движения.СтоимостьМатериалов.Записать();
  Движения.ОстаткиМатериалов.Записать();

  РезультатЗапроса = Запрос2.Выполнить();

  // ТЗ = РезультатЗапроса.Выгрузить();

  ВыборкаДетальныеЗаписи = РезультатЗапроса.Выбрать();

  Пока ВыборкаДетальныеЗаписи.Следующий() Цикл
    Если ВыборкаДетальныеЗаписи.Количество = 0 Тогда
      СтоимостьМатериала = 0;

    Иначе
      СтоимостьМатериала = ВыборкаДетальныеЗаписи.Стоимость /
      ВыборкаДетальныеЗаписи.Количество;

    КонецЕсли;

    Если ВыборкаДетальныеЗаписи.ВидНоменклатуры =
    Перечисления.ВидыНоменклатуры.Материал Тогда


      // Регистр ОстаткиМатериалов Расход
      Движение = Движения.ОстаткиМатериалов.Добавить();
      Движение.ВидДвижения = ВидДвиженияНакопления.Расход;
      Движение.Период = Дата;
      Движение.Материал = ВыборкаДетальныеЗаписи.Номенклатура;
      Движение.Склад = Склад;
      Движение.Количество = ВыборкаДетальныеЗаписи.КоличествоВДокументе;

      // Регистр СтоимостьМатериалов Расход
      Движение = Движения.СтоимостьМатериалов.Добавить();
      Движение.ВидДвижения = ВидДвиженияНакопления.Расход;
      Движение.Период = Дата;
      Движение.Материал = ВыборкаДетальныеЗаписи.Номенклатура;
      Движение.Стоимость = ВыборкаДетальныеЗаписи.КоличествоВДокументе *
      СтоимостьМатериала;
    КонецЕсли;

    // Регистр Продажи
    Движение = Движения.Продажи.Добавить();
    Движение.Период = Дата;
    Движение.Номенклатура = ВыборкаДетальныеЗаписи.Номенклатура;
    Движение.Клиент = Клиент;
    Движение.Мастер = Мастер;
    Движение.Количество = ВыборкаДетальныеЗаписи.КоличествоВДокументе;
    Движение.Выручка = ВыборкаДетальныеЗаписи.СуммаВДокументе;
    Движение.Стоимость = СтоимостьМатериала *
    ВыборкаДетальныеЗаписи.КоличествоВДокументе;

  КонецЦикла;

  Движения.Записать();
  #КонецОбласти

  #Область КонтрольОстатков
  Если Режим = РежимПроведенияДокумента.Оперативный Тогда
    // Проверить отрицательные остатки
    Запрос3 = Новый Запрос;
    Запрос3.МенеджерВременныхТаблиц = МенеджерВТ;
    Запрос3.Текст = "ВЫБРАТЬ
      | ОстаткиМатериаловОстатки.Материал КАК Материал,
      | ОстаткиМатериаловОстатки.КоличествоОстаток КАК КоличествоОстаток
      |ИЗ
      | РегистрНакопления.ОстаткиМатериалов.Остатки(
      | ,
      | Материал В
      | (ВЫБРАТЬ
      | НоменклатураДокумента.Номенклатура
      | ИЗ
      | НоменклатураДокумента)
      | И Склад = &Склад) КАК ОстаткиМатериаловОстатки
      |ГДЕ
      | ОстаткиМатериаловОстатки.КоличествоОстаток < 0";

    Запрос3.УстановитьПараметр("Склад", Склад);

    РезультатЗапроса = Запрос3.Выполнить();
    ВыборкаДетальныеЗаписи = РезультатЗапроса.Выбрать();

    Пока ВыборкаДетальныеЗаписи.Следующий() Цикл
      Сообщение = Новый СообщениеПользователю();
      Сообщение.Текст = "Не хватает " + Строка(-
      ВыборкаДетальныеЗаписи.КоличествоОстаток) + " единиц материала """ +
      ВыборкаДетальныеЗаписи.Материал + """";
      Сообщение.Сообщить();

      Отказ = Истина;

    КонецЦикла;

  КонецЕсли;
  #КонецОбласти

КонецПроцедуры
```

В результате мы можем свернуть выделенные программные области в тексте процедуры (рис. 17.10).

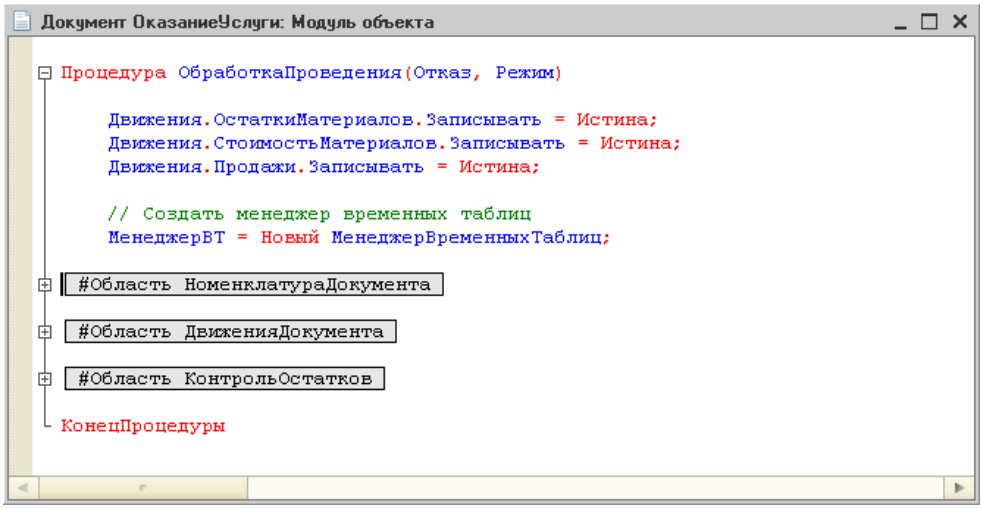


Рис. 17.10. Выделение произвольных областей в модуле

Затем можно развернуть только нужную область модуля и работать с ней. Текст процедуры станет более компактным и читаемым.

В нашем примере мы разбили одну процедуру модуля на три логические части. Но чаще наоборот: когда модуль содержит много различных процедур, сходные по назначению процедуры можно объединить в группы. Например, в модуле формы можно выделить такие области, как **ПрограммныйИнтерфейс**, **ОбработчикиСобытий**, **СлужебныеПроцедурыИФункции**. Название этих областей задается самим разработчиком и говорит само за себя.

Области могут быть вложены друг в друга или в другие группируемые конструкции языка. При расстановке областей в модуле нужно следить за тем, чтобы области не пересекались между собой и с другими группируемыми конструкциями. Потому что группировка по таким областям работать не будет.

- ***В режиме «1С:Предприятие»***

Запустим «1С:Предприятие» в режиме отладки и проверим работу нового обработчика события **ОбработкаПроведения**. Перепроведите все документы **Оказание услуги**.

В результате все работает точно так же, с точки зрения пользователя, но проведение документов организовано методически правильно и более эффективно с точки зрения доступа к данным.

Контрольные вопросы

1. Как система «1С:Предприятие» выполняет обращение к ссылочным данным?
2. Почему для доступа к массивам данных информационной базы предпочтительнее использовать запросы?
3. Что такое момент времени?
4. Чем отличается оперативное проведение документов от неоперативного?
5. Что такое оперативная отметка времени?
6. Как запросом получить остатки регистра накопления?
7. На что следует обращать внимание при указании параметров виртуальных таблиц запросов?
8. Почему при неоперативном проведении документов не нужно контролировать остатки?
9. Что такое временные таблицы, и зачем их использовать?
10. Что такое менеджер временных таблиц?
11. Как и зачем можно использовать временные таблицы в параметрах виртуальных таблиц?
12. Как программно блокировать данные?
13. Как выделить произвольные области в тексте программного модуля?